工具和模型都不换，只换代理循环：自建循环逐步用例打平，开放任务上通用循环赢在出错后补救

Volvo 车机 GUI 测试：生产中的**自建 harness**（固定四步 ReAct）与 **Claude Code / OpenCode** 共用同一工具层和设备接入，用同一模型跑。66 个逐步车机用例上成功率都在 **55–61%**（同模型差 −4.5 到 −0.8 点，p≥0.17）；Google Maps 开放任务上通用循环高 **+9 到 +28 点**，而它的单步工具正确率反而更低。代价是同模型下每题推理费高 **2.6–10.7 倍**（车机）。

01 · 剖面：两座楼，一块地基

Setup A · 自建 harness（生产在用，按 GPT-5.1 与车机调过）

① 观察 → ② 推理+清单 → ③ 校验 → ④ 执行

每轮恰好一个动作；harness 每步强制截图 + 精简 XML 层级

- 每步**从头重建 prompt**：历史只剩步骤日志 + 清单，截图只留上一个动作前后两张
- 维护：循环、prompt、上下文构造全由 Volvo 团队负责，换模型就要重调

Setup B · 通用 harness（Claude Code / OpenCode）

通用循环
何时看、何时停
由模型决定

- 上下文**累积**每次工具调用及其返回（含动作前后层级）
- 过长时由 harness 自动压缩，时机团队控制不了
- 适配只靠一份通用指令模板（所有用例相同）
- 维护：循环与上下文管理由厂商随新模型更新；Volvo 只维护指令模板、MCP 服务、工具描述与工具

本地 Python 调用　　经 MCP 服务调用

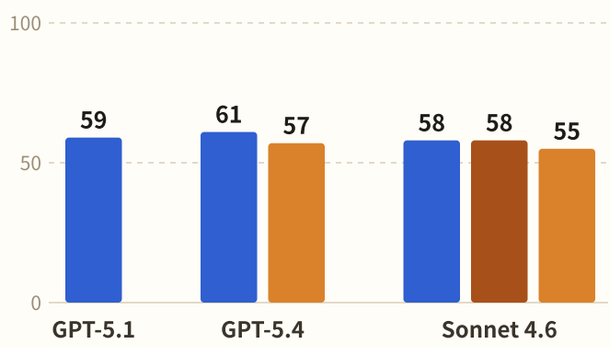
同一块地基（同一份代码）：共享工具层 → uiautomator2 → ADB → 车机 / 手机

设备工具（不调 LLM）：截图、XML 层级、点击、滑动、拖动、滑块、Home/Back，坐标由元素框算出
子代理（调 LLM）：滚动找项、输入/清空文本、数字选择器、等待条件 —— 两边相同；差异只剩策略 π 与编排（§III-C/D）

02 · 分户账：同一模型，三个场景

Volvo 车机 IVI

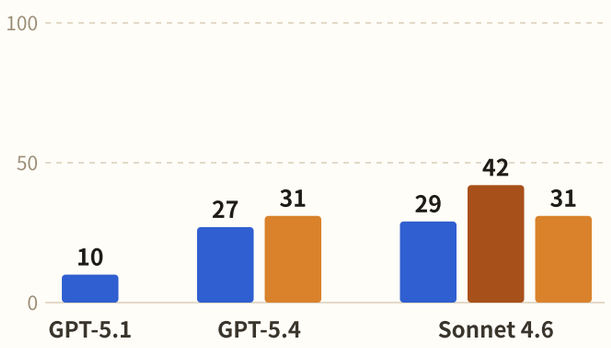
66 题 · 每配置 2 轮 · 逐步用例 · 测试台



同模型 B−A：−4.5 ~ −0.8 点，p≥0.17
flaky　A 2–3% · B 5–9%
每题费用　A $0.13–0.46 · B $0.92–4.97（同模型 2.6–10.7×）

Booking.com

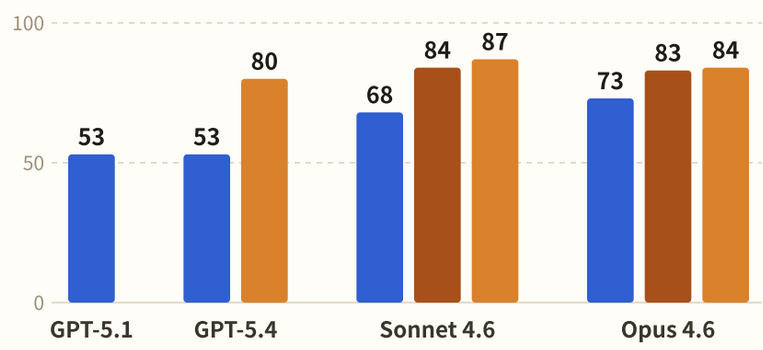
16 题 · 每配置 3 轮 · 长表单



B 同模型领先，16 题上不显著；里程碑达成 72–78% vs 56–59%
flaky　A 6–38% · B 19–50%
每题费用　A $1.41–3.53 · B $9.06–13.49

Google Maps

40 题 · 每配置 3 轮 · 开放目标



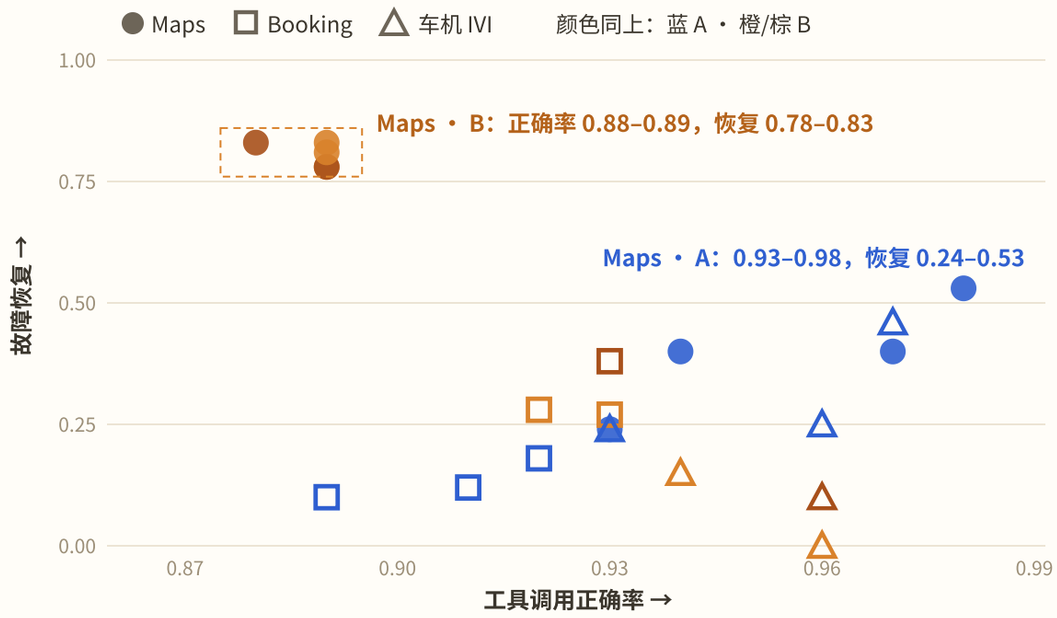
GPT-5.4 +28、Sonnet +17~19（p≤0.01）；Opus +9~11（p=0.04/0.06）
flaky　A 35–48% · B 13–25%
每题费用　A $0.40–2.40 · B $2.34–6.86

Setup A 自建　Setup B · Claude Code（仅 Anthropic 模型）　Setup B · OpenCode　　柱高 = 任务成功率（%）

成功率：Fig. 2（图中整数百分比）；差值与显著性：§IV-B（精确配对 Wilcoxon）；flaky = 重复运行间结果不一致的任务比例，Table I；成本：Table II（含子代理 LLM 调用，按实验时价格）

03 · 机制与总账

不是更准，是更会补救　Table I · 每点一个配置



“故障恢复”= 至少有一次工具调用被判错的运行中，最后仍成功的比例。作者把 B 的优势归因于累积的上下文能看出动作没生效并改正；这一归因没有做单独消融。车机用例逐步规定路径，错一步就偏离规定流程，两边恢复都低。

算总账：推理费换工程人力　§ IV-D · § VI · 估算

每月：工程人力（FTE）



B 多出的推理费按每个工作夜 60 个车机用例折算，低于省下的 0.4 FTE

合计月成本（工程 + 推理），B 相对 A：

−3% ~ −66%

OpenCode 省得最多，Claude Code 最少
对照：A 用生产模型 GPT-5.1，$0.13/题

若是大规模、高频跑的逐步回归集，推理费可能反过来占主导（§ VI）

工程投入（0.5 → 0.1 FTE）来自团队经理估计；“每晚 60 个车机用例”是假设的负载；价格按实验时报价。这是单一企业条件下的账，不是随机对照实验。延迟上 B 并不更慢：同模型每题平均少 3–57 秒。

Table II; §IV-D; §VI “Trading engineering effort for inference cost”

04 · 访谈：10 位从业者（4 开发、1 产品负责人、5 测试）· §V-B

F1 生态带来的技术债
自建系统没变坏；是新模型、新需求、成熟的通用 harness 让继续维护它越来越不划算

F2 维护转向集成
B 的工作集中在工具描述、参数设计和指令模板；A 每加一种能力都要重新评测整个循环

F3 失去上下文控制
开发者认为最大的损失是上下文管理与压缩时机不再由团队决定；成本是 B 评分最差的一项

F4 测试要看得见、守流程
测试员最看重可见性；被测应用崩溃时代理应停下报告，不能绕过去把缺陷藏住

限制与证据边界

- 成功由 LLM judge（GPT-5.6-sol）读完整轨迹判定；人工核对只做了 Booking 的一轮重复（无歧义的 88% 全部一致），车机和 Maps 没有人工核对。
- A 是按 GPT-5.1 和车机应用调过的，换到 AMEX 时“应用变了”同时伴随“逐步用例→开放目标”“测试台→手机”，作者也说不能单归因于应用。
- 车机只跑 2 轮、Booking 只有 16 题（差异不显著）；每题开始只回到主屏并启动应用，不是完整业务状态重置。
- 3 位开发者访谈对象（D2–D4）也是作者，参与搭建两套系统并跑实验；访谈评分未作测量报告。

这改变什么（我们的阅读）

- “专门写的固定循环一定更稳更好”在这里只对**逐步规定的用例**成立：成功率打平、结果更一致（flaky 2–3% vs 5–9%）。
- 开放目标上，收益来自**出错后的补救**而不是少犯错；比较 harness 时要同时报告单步正确率与故障恢复。
- 选 build 还是 adopt 是成本结构的取舍：固定的工程人力 vs 每次运行都付的推理费；测试方还要求代理在被测应用出故障时停下报告，而不是绕过去。
- 证据等级：**online**（真机交互，LLM judge 部分人工核验）。柱、点、散点由我们按原文表图重绘；剖面图为示意。